# Weekend Build Spec — DocChat API (RAG demo)

**What:** A Spring Boot service that ingests documents (PDF/text), embeds them into ChromaDB, and answers questions over them with source citations. **Why:** It's the exact shape of the AI-integration jobs we're seeing on Upwork (the OpenAPI/RAG MVP, the OpenAI API job) — and it turns "I've integrated OpenAI APIs" from a claim into a link you paste in proposals. **Time:** ~5–6 hours across a weekend.

---

## Stack

- Java 17, Spring Boot 3
- langchain4j (OpenAI chat model + embeddings) — what clients are asking for
- ChromaDB in Docker (vector store)
- Document parsing: langchain4j parsers / Apache PDFBox
- springdoc-openapi (the demo ships its own OpenAPI spec — nice flex)
- Docker Compose: app + ChromaDB, one command up
- JUnit 5 tests on ingestion + chat services

## Endpoints

| Method | Path | Does |
|---|---|---|
| POST | `/api/documents` | Upload PDF/text → parse, chunk, embed, store |
| GET | `/api/documents` | List ingested documents |
| POST | `/api/chat` | `{ "question": "..." }` → answer + cited source chunks |
| DELETE | `/api/documents/{id}` | Remove a document and its embeddings |

## Build sequence

1. **Hour 1 — Scaffold.** Spring Boot project, Docker Compose with ChromaDB, `/actuator/health` up. Commit.
2. **Hour 2 — Ingestion.** Upload endpoint: parse → split into chunks (~500 tokens, overlap) → embed via OpenAI → store in ChromaDB with metadata (doc name, chunk index).

3. **Hours 3–4 — Chat.** Question → embed → retrieve top-k chunks → prompt the chat model with context → return answer + citations. Handle the "not in the documents" case honestly (model says so instead of inventing).
4. **Hour 5 — Tests + hardening.** Unit tests for chunking and retrieval, error handling for bad uploads / empty store, README quickstart.
5. **Hour 6 — Polish.** Postman collection, architecture diagram in README (one mermaid block is fine), push to GitHub, pin the repo.

## Repo checklist

- [ ] README: what it does, architecture, quickstart (`docker compose up`), sample curl calls, screenshots/GIF of a Q&A
- [ ] Docker Compose brings up app + ChromaDB
- [ ] Postman collection committed
- [ ] Tests green, coverage badge optional
- [ ] `.env.example` for the OpenAI key — never commit a real key

---

## Showcase copy (ready to paste)

**Upwork portfolio title:** DocChat API — RAG Document Q&A Service (Spring Boot + LangChain4j)

**Upwork portfolio description:** Production-style retrieval-augmented generation service built with Java 17, Spring Boot 3, and LangChain4j. Upload PDF/text documents; the service chunks and embeds them into ChromaDB and answers questions with cited sources. Includes a REST API with OpenAPI docs, Docker Compose deployment, Postman collection, and unit tests. Demonstrates: LLM API integration, embeddings/vector search, prompt design with grounded answers, and clean enterprise Java delivery.

**Proposal snippet (for AI-integration jobs):**

> On the AI side, I built a RAG document-Q&A service with Spring Boot + LangChain4j and ChromaDB — ingestion, embeddings, retrieval, and cited answers — happy to share the repo: [GitHub link]. The engineering pattern (clean API layer, error handling, tests, Docker deployment) is the same one I'd bring to your project.

**Skill tags to add once it's live:** LangChain4j, Spring AI, OpenAI API, Vector Database, Retrieval-Augmented Generation (RAG), Docker Compose.